Unidad 2
Métodos de búsqueda

1. Búsquedas (en anchura y en produndidad)
2. Árboles de decisión
3. Teoria de juegos
4. Satisfacción de restricciones

Agentes

Es el concepto fundamental sobre el que se construye toda la Inteligencia Artificial moderna según el **libro de Russell y Norvig**. No podemos hablar de búsquedas, árboles de decisión o juegos sin antes entender quién es el "sujeto" que realiza todas estas acciones.

Un **agente** es cualquier cosa que puede ser considerada como **percibiendo su entorno a través de sensores** y **actuando sobre ese entorno a través de actuadores**.

La clave aquí es el ciclo percepción-acción. Un agente no es solo un programa; es una entidad que interactúa con un mundo, ya sea físico o virtual. Esta definición es tan amplia que cubre desde un termostato hasta un robot humanoide, pasando por un programa de ajedrez o un "chatbot".

El diagrama fundamental:

Agente = Arquitectura + Programa

- **Arquitectura:** Es la "máquina" física o virtual donde se ejecuta el programa. Puede ser un computador con cámaras y ruedas (un robot), o simplemente el hardware de un servidor (un agente de software).
- **Programa del Agente:** Es la "mente" del agente. Es la función que implementa la lógica de decisión, mapeando las percepciones a las acciones.

Los Componentes Clave de un Agente

1. Percepción (Sensores)

Es la entrada de información del mundo. Los sensores pueden ser:

- **Físicos:** Cámaras, micrófonos, sensores de tacto, GPS, sonar (para robots).
- **Lógicos/Software:** Entrada de teclado, archivos leídos, paquetes de red, bases de datos (para softbots o agentes de software como el de compras por Internet del Cap. 10).

Los Componentes Clave de un Agente

2. Acción (Actuadores)

Es la salida del agente que modifica el entorno. Los actuadores pueden ser:

- **Físicos:** Motores, ruedas, brazos robóticos, pinzas.
- **Lógicos/Software:** Escribir en un archivo, enviar un correo electrónico, mostrar un resultado en pantalla, o simplemente generar la siguiente palabra en una conversación.

Los Componentes Clave de un Agente

3. Función del Agente vs. Programa del Agente

El libro hace una distinción crítica (Capítulo 2, página 38):

- **Función del Agente:** Es una descripción matemática abstracta. Es una tabla (ideal) que mapea **cada posible secuencia de percepciones** a una **acción**. Es la especificación de cómo debería comportarse un agente ideal.
- **Programa del Agente:** Es la implementación concreta de esa función en una arquitectura real. Es el código que se ejecuta.

¿Qué hace que un Agente sea Bueno o Inteligente? La Racionalidad

- Tener un agente no es suficiente; queremos que sea **racional** (Cap 2, pág 40). Un **agente racional** es aquel que hace "lo correcto" para maximizar su medida de rendimiento.
- Un agente racional no es omnisciente (no lo sabe todo), sino que actúa basándose en lo que **espera** que ocurra, dadas sus percepciones y su conocimiento.

Definición de Racionalidad (del libro):

Para cada posible secuencia de percepciones, un agente racional debe realizar la acción que **se espera que maximice su medida de rendimiento**, basándose en la evidencia proporcionada por la secuencia de percepciones y en cualquier conocimiento previo que tenga.

Esto depende de cuatro factores:

1. La **medida de rendimiento** (qué define el éxito).
2. El **conocimiento previo** que tiene el agente sobre el entorno.
3. Las **acciones** que el agente puede realizar.
4. La **secuencia de percepciones** que ha tenido hasta el momento.

Tipos de Agentes (Estructuras Básicas)

El libro (Cap 2.4) describe una jerarquía de agentes, cada uno más complejo y flexible que el anterior. Estos son los diseños que implementan el programa del agente:

1. **Agente Reactivo Simple (Reflexivo)**
2. **Agente Reactivo Basado en Modelos**
3. **Agente Basado en Objetivos (Metas)**
4. **Agente Basado en Utilidad**
5. **Agente que Aprende**

1. Agente Reactivo Simple (Reflexivo)

- **Cómo funciona:** Selecciona acciones basándose *solo* en la percepción actual. Es un mapeo directo de condición-acción (reglas "si-entonces").
- **Ejemplo:** El agente aspiradora que aspira si ve suciedad, y se mueve si ve que está limpio.
- **Limitación:** Solo funciona si el entorno es totalmente observable. Si es parcialmente observable, puede caer en bucles infinitos.

2. **Agente Reactivo Basado en Modelos**

- **Cómo funciona:** Mantiene un **estado interno** que refleja los aspectos del mundo que no puede ver directamente. Este estado se actualiza con las percepciones y las acciones realizadas.
- **Ejemplo:** Un robot que recuerda que la puerta a su izquierda estaba cerrada en el último fotograma, así que no intenta pasar por allí.
- **Ventaja:** Puede manejar entornos parcialmente observables.

3. **Agente Basado en Objetivos (Metas)**

- **Cómo funciona:** Además del estado interno, tiene información sobre una **meta** (un estado deseado). Elige acciones que lo acerquen a esa meta, a menudo planificando secuencias de acciones (como los algoritmos de búsqueda que vimos).
- **Ejemplo:** El agente del mapa de Rumanía que busca una ruta a Bucarest.
- **Ventaja:** Es más flexible. Si el objetivo cambia, el comportamiento se adapta automáticamente sin tener que reescribir todas las reglas.

4. **Agente Basado en Utilidad**

- **Cómo funciona:** Tiene una función de **utilidad** que asigna un valor numérico a cada estado. Elige la acción que maximiza la utilidad esperada, incluso si no hay un único objetivo binario.
- **Ejemplo:** El taxista automático que debe equilibrar velocidad, seguridad, comodidad y consumo de combustible.
- **Ventaja:** Puede manejar objetivos en conflicto y decidir racionalmente bajo incertidumbre (la base de la teoría de la decisión).

5. **Agente que Aprende**

- **Cómo funciona:** Tiene un componente de aprendizaje que mejora el rendimiento del agente con la experiencia. Puede aprender nuevos modelos, nuevas políticas, o nuevas funciones de utilidad.
- **Ejemplo:** TD-Gammon (el jugador de backgammon) que aprendió a jugar mejor al jugar millones de partidas contra sí mismo.

1: Búsquedas (Anchura y Profundidad)

El Problema: Un agente se encuentra en un estado inicial y debe encontrar una secuencia de acciones que lo lleve a un estado objetivo. El espacio de todos los estados posibles puede ser representado como un grafo. La búsqueda es el proceso de explorar este grafo para encontrar un camino solución.

1.1 Búsqueda en Anchura (BFS - Breadth-First Search)

- **Esencia:** Explora el grafo por niveles. Primero expande todos los nodos a una profundidad, luego todos los nodos a la siguiente, y así sucesivamente. Es como una onda expansiva desde el punto de inicio.
- **Estructura de datos:** Utiliza una **cola (FIFO - First In, First Out)**. Los nodos que se descubren primero se exploran primero.

1: Búsquedas (Anchura y Profundidad)

- **Propiedades (según el Capítulo 3 de tu libro):**
  - **Completo:** Si existe una solución, BFS la encontrará, siempre que el factor de ramificación sea finito.
  - **Óptimo:** Sí, siempre y cuando el costo de cada acción sea el mismo (o el costo del camino sea una función no decreciente de la profundidad).
  - **Complejidad temporal:** O(b^(d+1)), donde **b** es el factor de ramificación (número máximo de sucesores de cualquier nodo) y **d** es la profundidad de la solución más superficial. Es exponencial.
  - **Complejidad espacial:** O(b^(d+1)). **Este es su principal punto débil**. Guarda todos los nodos generados en la frontera, lo que lo hace impracticable para problemas grandes.
- **Cuándo usarlo:** Ideal para problemas donde la solución está cerca de la raíz y el espacio de búsqueda no es demasiado grande (el mapa de Rumanía para encontrar la ruta más corta entre ciudades, por ejemplo). El libro lo menciona como una estrategia no informada (a ciegas, fuerza bruta), ya que no utiliza ninguna información adicional sobre el problema.

1: Búsquedas (Anchura y Profundidad)

1.2 Búsqueda en Profundidad (DFS - Depth-First Search)

- **Esencia:** Explora un camino hasta el final (una hoja o un callejón sin salida) y luego retrocede (*backtracking*) para explorar el siguiente camino. Es como moverse por un laberinto sin memoria de los caminos alternativos.
- **Estructura de datos:** Utiliza una **pila (LIFO - Last In, First Out)**. Se implementa fácilmente con recursión.
- **Propiedades:**
  - **Completo:** No. Puede caer en un bucle infinito si no se tiene cuidado (por ejemplo, un grafo con ciclos). Se puede volver completo usando un límite de profundidad.
  - **Óptimo:** No. No garantiza encontrar la ruta más corta; simplemente encuentra una solución.

1: Búsquedas (Anchura y Profundidad)

  - **Complejidad temporal:** O(b^m), donde **m** es la profundidad máxima de cualquier nodo en el árbol de búsqueda. En el peor caso, explora todo el árbol.
  - **Complejidad espacial:** O(bm). Su gran ventaja es que solo necesita almacenar el camino actual y los nodos hermanos no expandidos. Es muy eficiente en memoria.
- **Variante Crucial: Búsqueda de Profundidad Limitada y Búsqueda con Profundidad Iterativa (IDDFS).** El libro destaca la IDDFS como una estrategia óptima y completa que combina la eficiencia espacial de DFS con la optimalidad de BFS. Incrementa el límite de profundidad de forma iterativa hasta encontrar una solución.

Para tu proyecto: Si tu agente debe navegar en un espacio de estados muy grande y la memoria es limitada (como en un robot explorador), DFS o IDDFS son tus aliados. Si necesitas la solución más corta en un espacio pequeño, BFS es la mejor opción.

1. La Formulación del Problema (El "Puente" hacia la Búsqueda)

Un agente basado en objetivos no puede simplemente "buscar" sin una estructura clara. Para implementar cualquier tipo de búsqueda (de los Capítulos 3 y 4), necesitas entender cómo se formula un problema. Te falta interiorizar estos cuatro componentes:

- **El Estado Inicial:** ¿Dónde comienza el agente? (Ej: En(Arad) para el problema de Rumanía).
- **La Función de Transición (o Sucesor):** Dado un estado y una acción, ¿cuál es el nuevo estado? **Esta es la clave.** Define el *espacio de estados* y el *factor de ramificación* (b). Sin ella, no puedes ni siquiera empezar a hablar de BFS o DFS.
- **El Test de Objetivo:** ¿Cómo sabe el agente que ha llegado a su meta?
- **El Coste del Camino:** ¿Cuánto "cuesta" cada acción? Esto es vital para elegir una solución óptima.

Lo que te falta: Entender que la eficiencia de un algoritmo de búsqueda (BFS, DFS, A*, etc.) depende directamente de la "calidad" de la formulación del problema (como viste con las 8-reinas, donde una mala formulación genera un espacio de estados exponencialmente más grande).

2: Árboles de Decisión

El Problema: Aprender una función que clasifique ejemplos en base a sus atributos. Es un método de **aprendizaje supervisado** (Capítulo 18).

2.1 Fundamentos Matemáticos (Capítulo 18)

- Un árbol de decisión divide el espacio de búsqueda en regiones más pequeñas, realizando preguntas sobre los atributos de los ejemplos. Para decidir qué atributo usar en cada nodo, se utiliza la **Ganancia de Información**, basada en la **Entropía**.
- **Entropía:** Mide el desorden o la incertidumbre de un conjunto de datos. Cuanto más mezclados están los ejemplos de diferentes clases, mayor es la entropía.
  $H(S) = - \Sigma_i p_i \log_2 p_i$ (donde p_i es la proporción de ejemplos de la clase i en el conjunto S).
- **Ganancia de Información (IG):** Mide la reducción esperada en la entropía después de dividir los datos según un atributo A.
  $IG(S, A) = H(S) - \Sigma_v (|S_v| / |S|) * H(S_v)$
  El algoritmo **ID3** (y sus sucesores como C4.5) elige el atributo con la mayor Ganancia de Información para ser la raíz del árbol.

2: Árboles de Decisión

2.2 Overfitting y Poda

- **Problema (Overfitting):** Si el árbol crece demasiado, puede "memorizar" los datos de entrenamiento, incluyendo el ruido, y fallar al clasificar nuevos ejemplos.
- **Solución (Poda):** Reducir el tamaño del árbol para mejorar su capacidad de generalización.
  - **Pre-poda:** Detener el crecimiento del árbol antes de que se ajuste perfectamente a los datos (por ejemplo, usando un umbral de Ganancia de Información mínimo).
  - **Post-poda:** Dejar que el árbol crezca completamente y luego eliminar las ramas que no aportan significativamente a su rendimiento.

2: Árboles de Decisión

2.3 Random Forests y la Potencia del Conjunto

- Para mejorar la precisión y reducir el overfitting, se pueden construir múltiples árboles de decisión y combinar sus predicciones, formando un **Random Forest**.
- Cada árbol se entrena con una muestra aleatoria de los datos y en cada nodo se considera un subconjunto aleatorio de atributos. La predicción final se obtiene por votación.

Para tu proyecto: Usa árboles de decisión cuando necesites **explicabilidad** (como en diagnósticos médicos). Si tu prioridad es la precisión bruta, los Random Forests o los métodos de *boosting* que también se mencionan en el Capítulo 18 suelen ser mucho más potentes.

2. El Aprendizaje y el Ruido (El "Puente" hacia los Árboles de Decisión)

Para construir un árbol de decisión (Capítulo 18), un agente necesita aprender de los datos. Pero aprender no es simplemente construir un árbol; es hacerlo de forma que realmente funcione en el mundo real. Por eso te falta entender dos conceptos críticos:

- **El Sobreajuste (Overfitting):** Es la "maldición" del aprendizaje. Si tu árbol es demasiado complejo, se adaptará perfectamente a los datos con los que fue entrenado, pero fallará estrepitosamente con datos nuevos. El árbol habrá "memorizado" en lugar de "aprendido".
- **La Poda y el Ruido:** Para combatir el sobreajuste, se necesita podar el árbol (eliminar ramas irrelevantes). El libro destaca que los datos del mundo real casi siempre tienen **ruido** (errores, excepciones). Un buen agente debe ser robusto a este ruido.

Lo que te falta: Entender que no basta con calcular la entropía o la ganancia de información. El verdadero arte del aprendizaje (como se describe en el Capítulo 18) está en decidir cuándo dejar de crecer el árbol para evitar el sobreajuste y mantener la capacidad de generalización.

3: Teoría de Juegos (IA Adversaria)

El Problema: Un agente debe tomar decisiones en un entorno donde hay otro agente (el oponente) que tiene objetivos opuestos. Se busca la estrategia óptima, suponiendo que el oponente también juega de forma óptima (Capítulo 6).

3.1 El Algoritmo Minimax

- Modela el juego como un árbol donde los niveles alternan entre el **Jugador MAX** (que quiere maximizar el resultado) y el **Jugador MIN** (que quiere minimizarlo).
- El algoritmo asigna un valor a cada nodo terminal (la utilidad del estado, como +1 para victoria, -1 para derrota).
- **Propagación:** Los valores de los nodos se propagan hacia arriba.
  - En un nodo MAX, el valor es el máximo de los valores de sus hijos.
  - En un nodo MIN, el valor es el mínimo de los valores de sus hijos.
- La decisión en el nodo raíz es la acción que lleva al hijo con el valor más alto.

3: Teoría de Juegos (IA Adversaria)

3.2 Poda Alfa-Beta (La Optimización Clave)

- Minimax puro es exponencialmente costoso. La poda alfa-beta mejora la eficiencia sin comprometer la optimalidad.
- Mantiene dos valores:
  - **α**: el valor de la mejor opción (más alta) que MAX ha encontrado para él.
  - **β**: el valor de la mejor opción (más baja) que MIN ha encontrado para él.
- **Regla de Poda:**
  - Si en un nodo MAX se encuentra un valor **v ≥ β**, se puede podar el resto de hijos, porque MIN nunca permitirá que MAX llegue a este nodo.
  - Si en un nodo MIN se encuentra un valor **v ≤ α**, se puede podar el resto de hijos.
- Con un buen orden de los movimientos, la poda alfa-beta puede reducir la complejidad de O(b^d) a O(b^(d/2)), permitiendo explorar el doble de profundidad.

3: Teoría de Juegos (IA Adversaria)

3.3 Juegos Estocásticos (Expectiminimax)

- Para juegos que incluyen azar (ej. Backgammon), se añaden nodos de **posibilidad**.
- El valor de un nodo de posibilidad es el promedio ponderado de sus hijos:
  Valor(nodo posibilidad) = Σ (Probabilidad(resultado) * Valor(hijo)).

Para tu proyecto: Usa Minimax con Alfa-Beta para juegos de turnos perfectos y deterministas (ajedrez, damas). Para juegos con azar, usa Expectiminimax. No olvides usar una **función de evaluación heurística** (Capítulo 6) para cortar el árbol a una profundidad manejable en juegos complejos.

3. El Entorno Multiagente y la Inutilidad del Mínimo Local (El "Puente" hacia la Teoría de Juegos)

Para entender por qué Minimax no es suficiente para juegos como el póker o el backgammon, necesitas captar los desafíos del entorno en el que se juega.

- **Incertidumbre:** Un agente en un juego de azar (backgammon) no puede calcular un árbol de decisión perfecto. Necesita nodos de posibilidad (que vimos en el módulo) y funciones de evaluación probabilísticas.
- **El Mínimo Local (y la Poda Alfa-Beta):** El libro no solo explica la poda alfa-beta como una mejora de eficiencia, sino como una respuesta a la naturaleza del entorno de juego (que es como un grafo de estados). Poda es la forma de "saltar" por encima de soluciones subóptimas sin tener que explorarlas.

Lo que te falta: Interiorizar que en un entorno multiagente, el "mejor movimiento" no existe en el vacío. Depende de lo que haga el otro agente. Esto cambia radicalmente el enfoque de la búsqueda (pasas de maximizar tu utilidad a minimizar la del oponente) y te obliga a conceptos como la **poda alfa-beta** y los **juegos estocásticos**.

4: Satisfacción de Restricciones (CSP - Constraint Satisfaction Problems)

El Problema: Dado un conjunto de **variables** (cada una con un **dominio** de valores posibles) y un conjunto de **restricciones** entre ellas, encontrar una asignación de valores a las variables que satisfaga *todas* las restricciones. Se busca una solución factible, no una óptima (Capítulo 5).

4.1 Definición Formal y Ejemplos

- **Variables:** $X = \{X_1, X_2, \ldots, X_n\}$
- **Dominios:** $D = \{D_1, D_2, \ldots, D_n\}$ (conjuntos de valores posibles, usualmente finitos y discretos).
- **Restricciones:** $C = \{C_1, C_2, \ldots, C_m\}$. Cada restricción involucra un subconjunto de variables y define las combinaciones de valores permitidos.
- **Ejemplos del libro:** El problema de colorear el mapa de Australia (variables: regiones; dominios: colores; restricción: regiones adyacentes no pueden tener el mismo color) y el problema de las 8-reinas.

4: Satisfacción de Restricciones (CSP - Constraint Satisfaction Problems)

4.2 Algoritmo de Backtracking con MRV

- El enfoque estándar para resolver un CSP es la búsqueda con **vuelta atrás (backtracking)**, que es un DFS que asigna valores a las variables de forma incremental.
- **Heurísticas Cruciales:**
  - **MRV (Mínimos Valores Restantes) (Capítulo 5):** "Falla rápido". Elige la variable con el dominio más pequeño (la que tiene menos valores posibles). Esto reduce drásticamente el factor de ramificación.
  - **Grado (Degree Heuristic):** Para desempatar, elige la variable que está involucrada en más restricciones con otras variables no asignadas.
  - **LCV (Valor Menos Restringido) (Capítulo 5):** Para ordenar los valores. Elige primero el valor que elimina menos valores de los dominios de las variables vecinas, maximizando la flexibilidad futura.

4: Satisfacción de Restricciones (CSP - Constraint Satisfaction Problems)

4.3 Propagación de Restricciones (Consistencia de Arco)

- **Consistencia de Arco (AC-3, Capítulo 5):** Un preprocesamiento que "propaga" las consecuencias de las restricciones para reducir los dominios.
- **Idea:** Si un valor del dominio de la variable X_1 no tiene un valor consistente en el dominio de la variable X_2 para una restricción entre ellas, ese valor se elimina del dominio de X_1.
- El algoritmo AC-3 mantiene una cola de arcos y los revisa, eliminando valores y agregando nuevos arcos a la cola si un dominio se reduce.
- **Resultado:** La búsqueda posterior es mucho más rápida, y a veces descubre que el CSP es insatisfacible antes de buscar.

4: Satisfacción de Restricciones (CSP - Constraint Satisfaction Problems)

4.4 Búsqueda Local para CSPs

- **Mínimos Conflictos (Capítulo 5):** Un algoritmo de búsqueda local muy eficiente para CSPs.
- **Idea:** Se parte de una asignación completa (aunque sea inválida) y se modifica iterativamente el valor de una variable para reducir el número de restricciones violadas.
- **Características:** Es sorprendentemente eficaz para problemas como las n-reinas y la programación de telescopios.

Para tu proyecto: Los CSPs son la herramienta ideal para problemas de planificación de horarios, asignación de recursos y configuración. Usa Backtracking con MRV y AC-3 para la mayoría de los casos. Si el problema es muy grande, considera Mínimos Conflictos.

4. La Estructura del Problema y el Razonamiento con Restricciones (El "Puente" hacia la Satisfacción de Restricciones)

A diferencia de los problemas de búsqueda generales, un CSP tiene una estructura declarativa muy clara. Pero su poder no viene de la estructura en sí, sino de cómo se explota. Aquí te falta entender dos ideas clave del Capítulo 5:

- **La Propagación de Restricciones (AC-3):** No es solo una técnica de optimización, es un cambio de paradigma. En lugar de "probar y fallar" (backtracking), el agente primero usa la lógica de las restricciones para reducir los dominios de las variables *antes* de buscar. Es como resolver un sudoku usando la lógica en lugar de la fuerza bruta.
- **La Estructura del Grafo (Descomposición):** Un CSP no es solo un conjunto de variables; es un grafo (como el mapa de Australia). El libro dedica una sección a mostrar que si el grafo de restricciones tiene una estructura de **árbol**, el problema se puede resolver en tiempo lineal. Esta es una lección poderosa: la complejidad del problema depende tanto de la estructura de las restricciones como del número de variables.

Lo que te falta: Entender que la clave de un CSP no es solo *encontrar* una solución, sino *estructurar* el problema de manera que la solución sea fácil de encontrar (usando heurísticas como MRV o la descomposición en árboles).